# Polymorphism

Polymorphism in C++ is one of the fundamental concepts of object-oriented programming (OOP). It allows a single function, operator, or class to operate in multiple ways. The terms "polymorphism" means "many forms", enabling flexibility and reusability in code.

→ Types of polymorphism in C++ :-

Polymorphism in C++ is mainly divided into two types :

1.) Compile-time polymorphism (static binding) :-

(i) function overloading :-

Function overloading allows multiple function with the same name but different parameters. The compiler determines which function to execute based on the argument passed.

Eg;

```
void show (string name) // same function name
{                       // different parameter
}
void show (int age)
{
}
```

Akash

→ // C++ eg. for overloading function;

```
#include <iostream>
using namespace std;
class student
{
    public:
        string name;
        int age;
        int roll;

    // function with string parameter

        void show (string name)
        {
            cout<<name <<endl;
        }

    // function with two integer parameter

        void show(int age, int roll)
        {
            cout<<age<<endl;
            cout<< roll;
        }
};

int main()
{
```

Akash

```
Student s;
s.show("akash kumar");   // function call
s.show(20, 12355);       // function call
return 0;
}
```

(ii) constructor overloading :

Constructor overloading is a concept where multiple constructors are defined in a class with the same name but different parameters. It allows the creation of objects in different ways.

Eg;

```
class Student
{

    Student()   default constructor
    {
        // body
    }

    Student(int age, int roll)   parameterized constructor
    {
        // body
    }
};
```

Akash

→ C++ program;

```
#include<iostream>
using namespace std;
class Student
{
    public:
    int age;
    int roll;

    Student() // default constructor
    {
        cout<<"constructor is called";
    }

    Student(int age, int roll) // parameterized const.
    {
        cout<<"age=" << age<<endl;
        cout<<"roll=" << roll<<endl;
    }
};

int main()
{
    Student s1; // calling default constructor
    Student s2(20,123555); // parameterized const.
    return 0;
}
```

Akash

## 2.> Run time polymorphism (Dynamic Binding)

### (i) Function overriding :-

Parent & child both contain the same function with different implementation. The parent class function is said to be overridden.

→ // C++ program;

```
#include<iostream>
using namespace std;
class parents   // base class
{
        public :
          void show()
           {
              cout<<"parents";
           }
};

class child : public parents // derive class
{
        public :
          void show()
           {
              cout<<" child";
           }
};
```

Akash

```
int main()
{
    parent p;
    p.show()      // calling base class
    child c;
    c.show();     // calling derive class
    return 0;
}
```

(ii) Virtual function :-

Virtual function are used to :

- Enable dynamic (runtime) polymorphism.
- Ensure that the correct overridden function is called for an object, regardless of the type of reference (or pointers) used.

→ // C program ;

```
#include<iostream>
using namespace std;
class parent
{
    public :
    virtual void show()
    {
        cout<<" parent class";
    }
};
```

Akash

```
child class : public parent
{
    public :
      void show()
      {
        cout<<"Child class";
      }
};

int main()
{
    parent *ptr;
    parent p;
    ptr= &p;
    ptr->show();
    child c
    ptr-> &c;
    ptr->show();
}
```

Akash